

Report di Progetto: Ingegneria del Software

GIANGREGORIO DANIELE - FORMICA RICCARDO
GALANTE ALESSANDRO - LA ROSA GIUSEPPE

1 Funzionalità implementate rispetto ai requisiti

Oltre alle funzionalità di base previste per il gioco, sono state implementate le seguenti Funzionalità Avanzate :

- **Resilienza (Gestione Disconnessioni):** Il sistema è progettato per tollerare la disconnessione dei client sia in fase di lobby che a partita avviata. Un giocatore disconnesso mantiene il vincolo sul colore del totem. Se la disconnessione lascia un solo giocatore in lobby, viene avviato un timer di 30 secondi che, in assenza di riconnessioni tempestive, decreta la vittoria a tavolino.
- **Persistenza:** Lo stato della partita viene salvato automaticamente al termine di ogni round. In caso di crash del server, i client vengono avvisati e restano in ascolto per la riconnessione tramite tentativi periodici crescenti. Al riavvio del server, ai client viene richiesto di confermare nome e totem; se tutti i partecipanti originari si riconnettono correttamente, il server ripristina e fa proseguire la partita a partire dall'ultimo round salvato.
- **Classifica Partita su DB:** I risultati finali delle partite vengono storicizzati su un Database Relazionale (tramite la classe `DatabaseManager`). Al termine di una partita, oltre al vincitore, viene presentata ai giocatori una classifica globale relativa alle partite disputate storicamente con lo stesso numero di giocatori.

2 Divisione del lavoro tra i componenti del gruppo

La suddivisione dei compiti all'interno del gruppo è stata la seguente:

- **Riccardo:** Game, Board, Controller (prima stesura), TUI, FA persistenza.
- **Giuseppe:** Card, Buildings, EventShaman, Rete RMI/Socket lato client, modifiche ai Controller per l'implementazione della FA resilienza.
- **Alessandro:** Player, OfferTile, TurnOrderTile, i Deck (tribe e building), CardFactory e i json, Socket lato server, RMI lato server, FA database.
- **Daniele:** Characters, restanti eventi, client interface e parte della server interface, GUI.

3 Scelte implementative e design pattern usati

L'architettura del sistema si basa sul pattern **MVC (Model-View-Controller)**, applicato in un contesto di rete distribuito:

- **Controller:** La logica di orchestrazione è divisa tra `GameController` (gestione della partita e persistenza), `TurnController` (macchina a stati per l'avanzamento dei turni e la gestione asincrona di piazzamento totem e risoluzione azioni) e `LobbyController` (fase di setup). Il Controller comunica con la View inviando `ServerEvent`.
- **View:** Sono state sviluppate due interfacce utente, `TUIView` (Testuale) e `GUIView` (Grafica in JavaFX), che implementano l'interfaccia comune `ViewInterface`. Entrambe reagiscono in modo asincrono agli eventi del server.
- **Model:** Il modello del gioco è costituito da classi che rappresentano gli oggetti del mondo di gioco (Player, Card, Building, etc.), mantenendo lo stato e definendo le operazioni possibili su di essi.

Sono stati inoltre adottati i seguenti **Design Pattern**:

- **Factory Method:** È implementato mediante la classe `CardFactory` al fine di ottenere un'istanza dinamica delle carte (Tribe, Building, Eventi), partendo dai file JSON di configurazione. Ciò permette che il `GameController` non abbia al suo interno informazioni a riguardo.

- **Singleton:** Utilizzato per l'accesso globale al `DatabaseManager`, garantendo un unico punto di accesso per il salvataggio dei risultati e il recupero delle classifiche.
- **Strategy Pattern:** Utilizzato per la composizione dei `Building` con i corrispettivi `BuildingEffect`, permettendo di definire comportamenti dinamici e intercambiabili per le azioni speciali degli edifici.
- **Command Pattern:** Utilizzato per incapsulare i messaggi di rete (nella parte di `Socket`) sia lato server (con il `Listener` che fa da `Invoker`) che lato client (con le `ClientOperation`), permettendo di definire un'interfaccia comune per l'invio e la ricezione (grazie alla `ViewInterface`) dei comandi ().

Per quanto riguarda la **gestione della concorrenza e dei thread**:

- **GUI (JavaFX):** Sfrutta estensivamente `Platform.runLater()` (fornito dal `getUIDispatcher()`) per assicurare che gli aggiornamenti visivi avvengano sul JavaFX Application Thread, evitando eccezioni concorrenti. I task in background (es. attesa connessione) usano thread separati sincronizzati tramite `CountDownLatch`.
- **TUI:** Utilizza un'architettura a thread multipli con `ExecutorService` dedicati per separare l'ascolto dell'input (`input-handler`) dalla visualizzazione degli eventi del server (`ui-display`). Un `LinkedBlockingQueue` funge da buffer per l'input utente, mentre il blocco `synchronized` su `System.out` previene l'accavallamento visivo tra prompt e aggiornamenti asincroni del gioco.

4 Protocolli di comunicazione

L'infrastruttura di rete supporta contemporaneamente due protocolli per la comunicazione Client-Server, garantendo la standardizzazione dei messaggi tramite interfacce comuni:

- **Socket (TCP):** La comunicazione avviene scambiando messaggi serializzati in formato JSON (con supporto al polimorfismo garantito da una `RuntimeTypeAdapterFactory`). Lato client, il main thread è protetto dai blocchi I/O grazie all'uso di un thread demone (`Listener`) che legge in background e inoltra i dati al Dispatcher della UI. Un meccanismo dedicato rileva tempestivamente errori di rete gestendo flussi di input nulli o eccezioni I/O.
- **RMI (Remote Method Invocation):** L'implementazione utilizza il pattern *Callback*, invertendo i ruoli: il Client si registra come ascoltatore (`VirtualView`) permettendo al Server di invocare metodi da remoto asincronamente. A differenza dei socket, l'invio non è serializzato su JSON ma sfrutta il polimorfismo dei metodi Java. Poiché RMI non fornisce stream attivi da poter monitorare per le disconnessioni, è stato implementato un sistema di Heartbeat (tramite `ScheduledExecutorService`) che effettua dei "ping" costanti per validare l'integrità della connessione tramite l'eventuale cattura di `RemoteException`.

5 Percentuali di copertura dei test

La scrittura dei test di unità si è concentrata maggiormente sui rami critici legati alle logiche di dominio (Model) e di orchestrazione del gioco (Controller), garantendo alta affidabilità sulle regole della partita.

Package	Class %	Method %	Line %	Branch %
<code>it.polimi.ingsw</code>	73% (76/103)	45% (291/639)	33% (1058/3180)	28% (336/1163)
<code>controller</code>	100% (3/3)	93% (43/46)	71% (285/398)	57% (127/222)
<code>model</code>	91% (43/47)	84% (205/244)	82% (569/689)	72% (195/269)
<code>network</code>	51% (20/39)	12% (22/179)	19% (139/724)	5% (12/214)
<code>persistence</code>	100% (3/3)	61% (8/13)	63% (21/33)	0% (0/2)
<code>database</code>	50% (1/2)	21% (3/14)	16% (14/85)	10% (2/20)
<code>userInterface</code>	0% (0/3)	0% (0/133)	0% (0/1221)	0% (0/436)